



Department of
Software Engineering
BAHRIA UNIVERSITY
Discovering Knowledge

Design & Analysis of Algorithm Lecture 1

Dr. Qamaruddin Memon (Assistant Professor)
Department of Software Engineering
Bahria University Karachi Campus

Course Information

- Course webpage
 - BUKC-LMS
- Grading policy
 - Assign. 20%, Quizzes.10% Midterm: 20%, Final: 50%

More Information

■ Textbook

- *Algorithms*. Jeff Erickson(2019) 1st ed.
- *Data Structures and Algorithm Analysis*. Clifford A. Shaffer, Edition 3.2 (C++ Version), 2012.

■ Others

- *Introduction to Design & Analysis Computer Algorithm 3rd*, Sara Baase, Allen Van Gelder, Adison-Wesley, 2000.
- *Algorithms*, Richard Johnsonbaugh, Marcus Schaefer, Prentice Hall, 2004.
- *Introduction to The Design and Analysis of Algorithms 2nd Edition*, Anany Levitin, Adison-Wesley, 2007.

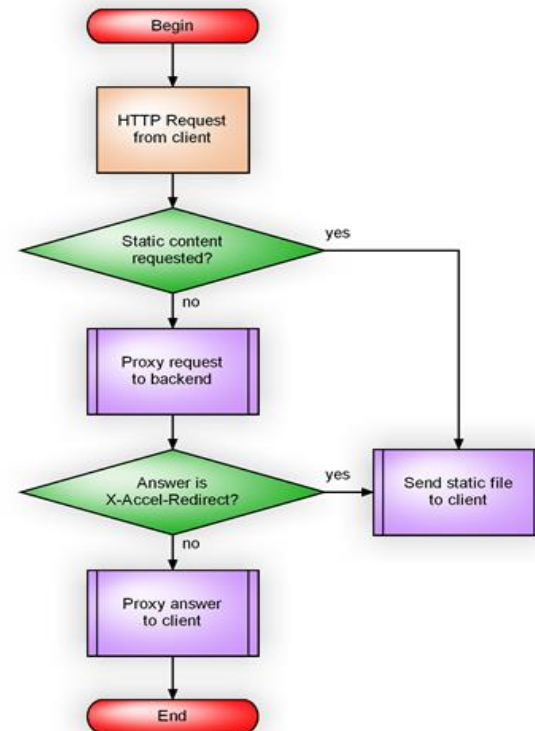
COURSE LEARNING OUTCOMES (CLOS):

Upon completion of this course, students will be able to:

CLO #	CLO Statements	Bloom's Taxonomy
CLO 1	Describe the basic concepts related to design and analysis of algorithms	C1
CLO 2	Explain the working mechanism of algorithmic design paradigms and approaches used to analyze those algorithms	C2
CLO 3	Design algorithms in pseudo-code of given problems	C3
CLO 4	Analyze the asymptotic performance of the algorithms	C4

Introduction

- What is an algorithm?
 - a step-by-step procedure to solve a problem
 - every program is the instantiation of some algorithm



http://blog.kovyryn.net/wp-content/uploads/2006/05/algorithm_c.png

Sorting Example



- Solves a general, well-specified problem
 - given a sequence of n keys, $a_1, \dots, a_n$, as input, produce as output a reordering $b_1, \dots, b_n$ of the keys so that $b_1 \leq b_2 \leq \dots \leq b_n$.
- Problem has specific instances
 - [Dopey, Happy, Grumpy] or [3,5,7,1,2,3]
- Algorithm takes every possible instance and produces output with desired properties
 - insertion sort, quicksort, heapsort, ...

Challenge

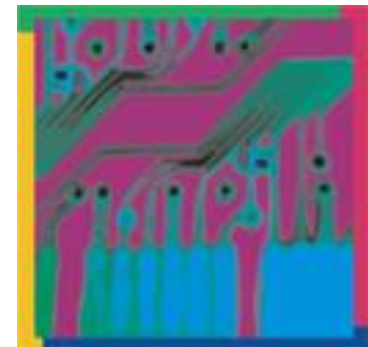
- Hard to design algorithms that are
 - correct
 - efficient
 - implementable
- Need to know about
 - design and modeling techniques
 - resources - don't reinvent the wheel

Correctness

- How do you know an algorithm is correct?
 - produces the correct output on every input
- Since there are usually infinitely many inputs, it is not trivial
- Saying "it's obvious" can be dangerous
 - often one's intuition is tricked by one particular kind of input

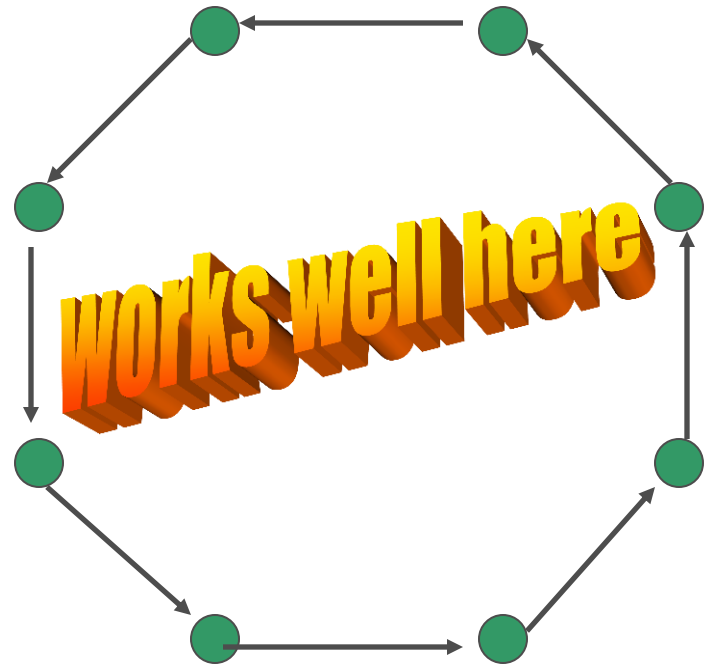
Tour Finding Problem

- Given a set of n points in the plane, what is the **shortest** tour that visits each point and returns to the beginning?
 - application: robot arm that solders contact points on a circuit board; want to minimize movements of the robot arm
- How can you find it?

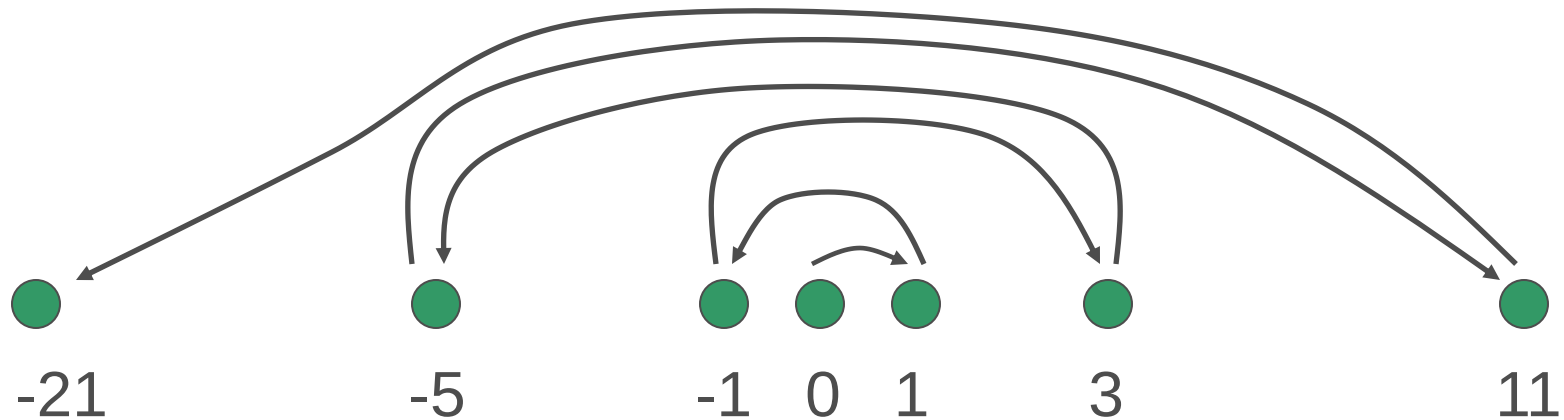


Finding a Tour: Nearest Neighbor

- start by visiting any point
- while not all points are visited
 - choose unvisited point closest to last visited point and visit it
- return to first point



Nearest Neighbor Counter-example



works poorly here

How to Prove Correctness?

- There exist formal methods
 - even automated tools
 - more advanced than this course
- In this course we will primarily rely on more informal reasoning about correctness
- Seeking counter-examples to proposed algorithms is important part of design process

Efficiency

- Software is always outstripping hardware
 - need faster CPU, more memory for latest version of popular programs
- Given a problem:
 - what is an efficient algorithm?
 - what is the most efficient algorithm?
 - does there even exist an algorithm?

How to Measure Efficiency

- Machine-independent way:
 - analyze "pseudocode" version of algorithm
 - assume idealized machine model
 - one instruction takes one time unit
- "Big-Oh" notation
 - order of magnitude as problem size increases
- Worst-case analyses
 - safe, often occurs most often, average case often just as bad

Faster Algorithm vs. Faster CPU

- A faster algorithm running on a slower machine will always win for large enough instances
- Suppose algorithm S1 sorts n keys in $2n^2$ instructions
- Suppose computer C1 executes 1 billion instruc/sec
 - When $n = 1$ million, takes 2000 sec
- Suppose algorithm S2 sorts n keys in $50n\log_2 n$ instructions
- Suppose computer C2 executes 10 million instruc/sec
 - When $n = 1$ million, takes 100 sec

Caveat

- No point in finding fastest algorithm for part of the program that is not the bottleneck
- If program will only be run a few times, or time is not an issue (e.g., run overnight), then no point in finding fastest algorithm

What do we analyze about them?

- Correctness

- Does the input/output relation match algorithm requirement?

- Amount of work done (aka complexity)

- Basic operations to do task finite amount of time

- Amount of space used

- Memory used

Strengthening the Informal Definition

■ Important Features:

- Finiteness.[algo should end in finite amount of steps]
- Definiteness.[each instruction should be clear]
- Input.[valid input clearly specified]
- Output.[single/multiple valid output]
- Effectiveness.[steps are sufficiently simple and basic]

RAM model

- has one processor
- executes one instruction at a time
- each instruction takes "unit time"
- has fixed-size operands, and
- has fixed size storage (RAM and disk).



Example Of Algorithm

Few Classical Examples

Classical Multiplication Algorithms

English

American

A la russe

Divide and Conquer

Classic Multiplication

CLASSIC MULTIPLICATION ALGORITHMS **(981 x 1234)**

981
1234

3924
2943
1962
981

1210554

American

981
1234

981
1962
2943
3924

1210554

English

Multiplication

MULTIPLICATION (981 x 1234) ***(a la russe algorithm)***

981	1234	1234
490	2468	
245	4936	4936
122	9872	
61	19744	19744
30	39488	
15	78976	78976
7	157952	157952
3	315904	315904
1	631808	<u>631808</u>
		<u>1210554</u>

Multiplication

MULTIPLICATION (981 x 1234)

(Divide-and-Conquer Algorithm)

	Multiply		Shift	Result
i)	09	12	4	108....
ii)	09	34	2	306..
iii)	81	12	2	972..
iv)	81	34	0	2754
				1210554

The Selection Problem

Which algorithm is better?



Which algorithm is better?

The algorithms are correct,

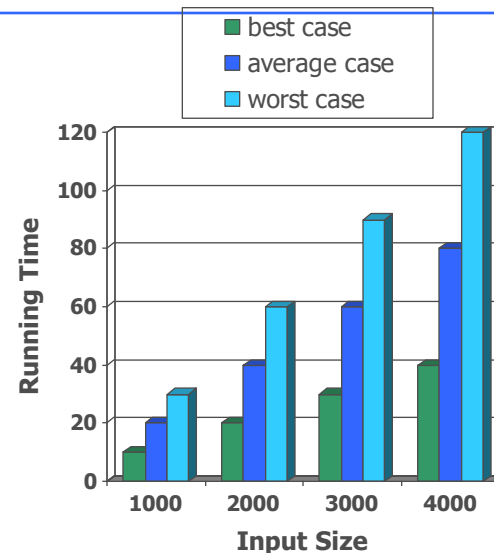
but which is the best?

- Measure the running time (number of operations needed).
- Measure the amount of memory used.
- Note that the running time of the algorithms increase as the size of the input increases.



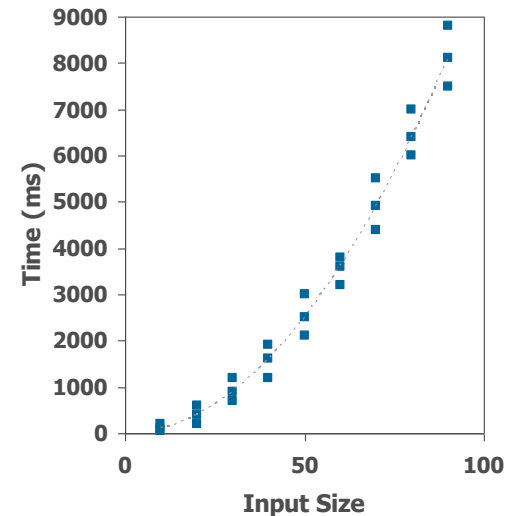
Running Time

- Most algorithms transform input objects into output objects.
- The running time of an algorithm typically grows with the input size.
- Average case time is often difficult to determine.
- We focus on the worst case running time.
 - Easier to analyze
 - Crucial to applications such as games, finance and robotics



Experimental Studies

- Write a program implementing the algorithm
- Run the program with inputs of varying size and composition
- Use a method like `System.currentTimeMillis()` to get an accurate measure of the actual running time
- Plot the results



Limitations of Experiments

- It is necessary to implement the algorithm, which may be difficult
- Results may not be indicative of the running time on other inputs not included in the experiment.
- In order to compare two algorithms, the same hardware and software environments must be used

Theoretical Analysis



- Uses a high-level description of the algorithm instead of an implementation
- Characterizes running time as a function of the input size, n .
- Takes into account all possible inputs
- Allows us to evaluate the speed of an algorithm independent of the hardware/software environment

Complexity

Time complexity:

- How much time it takes to compute
- Measured by a function $T(N)$

Space complexity:

- How much memory it takes to compute
- Measured by a function $S(N)$

Modeling the Real World

- Cast your application in terms of well-studied abstract data structures

<i>Concrete</i>	<i>Abstract</i>
arrangement, tour, ordering, sequence	permutation
cluster, collection, committee, group, packaging, selection	subsets
hierarchy, ancestor/descendants, taxonomy	trees
network, circuit, web, relationship	graph
sites, positions, locations	points
shapes, regions, boundaries	polygons
text, characters, patterns	strings

Real-World Applications

- Hardware design: VLSI chips
- Compilers
- Computer graphics: movies, video games
- Routing messages in the Internet
- Searching the Web
- Distributed file sharing
- Computer aided design and manufacturing
- Security: e-commerce, voting machines
- Multimedia: CD player, DVD, MP3, JPG, HDTV
- DNA sequencing, protein folding
- and many more!

Some Important Problem Types

- Sorting
 - a set of items
- Searching
 - among a set of items
- String processing
 - text, bit strings, gene sequences
- Graphs
 - model objects and their relationships
- Combinatorial
 - find desired permutation, combination or subset
- Geometric
 - graphics, imaging, robotics
- Numerical
 - continuous math: solving equations, evaluating functions

Algorithm Design Techniques

- Brute Force & Exhaustive Search
 - follow definition / try all possibilities
- Divide & Conquer
 - break problem into distinct subproblems
- Transformation
 - convert problem to another one
- Dynamic Programming
 - break problem into overlapping subproblems
- Greedy
 - repeatedly do what is best now
- Iterative Improvement
 - repeatedly improve current solution
- Randomization
 - use random numbers